Faculdade de Design, Tecnologia e Comunicação
Universidade Europeia

WHOPLAY

DESENVOLVIMENTO DE UMA APLICAÇÃO MÓVEL

PROJETO MULTIDISCIPLINAR

RELATÓRIO DA 2.ª ENTREGA

Grupo: 06
Elementos:

- **20251285 Cristiano Cordeiro**
- **20250953 Hélio José**
- **20251225 Sebastião Cardoso**
- **20252440 Tchali Tchalimba**

Git: https://github.com/tchaliuziel/WhoPlay-app.git

ÍNDICE

INTRODUÇÃO 1

1. PONTO DE SITUAÇÃO 1

2. REQUISITOS IMPLEMENTADOS 2

2.1. REQUISITOS FUNCIONAIS 2

2.2. REQUISITOS NÃO FUNCIONAIS 2

2.3. FUNCIONALIDADES ADICIONAIS 3

3. ARQUITETURA IMPLEMENTADA 3

4. BASE DE DADOS 3

4.1. TABELAS 4

4.2. SCRIPTS SQL 4

5. API REST 4

5.1. MÓDULOS 5

5.2. PRINCIPAIS ENDPOINTS 5

6. APLICAÇÃO MÓVEL 5

6.1. NAVEGAÇÃO 5

6.2. MELHORIAS E CORREÇÕES 6

7. MODELO DE COMPATIBILIDADE 6

8. TESTES E VALIDAÇÃO 6

9. PRÓXIMOS PASSOS 7

CONCLUSÃO 8

INTRODUÇÃO

A **WhoPlay** é uma aplicação móvel que ajuda as pessoas a encontrarem com quem praticar desporto, considerando a modalidade, o nível de experiência, a disponibilidade e a proximidade geográfica.

Na primeira entrega, o grupo apresentou o enquadramento do projeto, os objetivos, o público-alvo, a análise da concorrência, os requisitos, os mockups e o planeamento. Este relatório, correspondente à segunda entrega, centra-se no trabalho realizado desde então: a passagem do projeto para uma **versão alfa funcional**.

A aplicação Flutter está ligada a uma API REST em Node.js/Express e a uma base de dados MySQL. Os fluxos centrais, nomeadamente a autenticação, a pesquisa de jogadores, a criação de jogos, a participação em jogos e a edição do perfil, foram testados com sucesso, e os dados testados mantêm-se disponíveis depois de a aplicação ser fechada e reaberta.

Ao longo do documento distingue-se o que foi **confirmado** em teste do que está **implementado, mas por validar**, de modo a retratar com rigor o estado atual do projeto.

1. PONTO DE SITUAÇÃO

A tabela seguinte resume a evolução de cada componente entre as duas entregas.

Componente	**1.ª entrega**	**2.ª entrega**
Base de dados	Prevista em MySQL	Base whoplay em funcionamento, com oito tabelas
Backend	Previsto em Node.js	API REST em Express, com sete módulos de rotas
Aplicação	Mockups e requisitos	Aplicação Flutter ligada à API, com cinco áreas de navegação
Matching	Conceito	Modelo de compatibilidade de 100 pontos documentado
Código	Repositório GitHub	Desenvolvimento em branch própria, com histórico de commits

2. REQUISITOS IMPLEMENTADOS

Os requisitos definidos na primeira entrega mantêm a numeração original, para facilitar a rastreabilidade.

2.1. REQUISITOS FUNCIONAIS

ID	Requisito	Estado
RF01	Criar conta	Confirmado
RF02	Iniciar sessão	Confirmado
RF03	Terminar sessão	Por validar
RF04	Editar o perfil	Confirmado
RF05	Adicionar modalidades ao perfil	Confirmado
RF06	Indicar o nível em cada modalidade	Confirmado
RF07	Definir a disponibilidade	Confirmado
RF08	Definir uma localização aproximada	Confirmado
RF09	Procurar jogadores	Confirmado
RF10	Calcular compatibilidade entre jogadores	Implementado, por validar

2.2. REQUISITOS NÃO FUNCIONAIS

ID	Requisito	Estado
RNF01–02	Flutter e Dart no frontend	Cumprido
RNF03	Backend em Node.js	Cumprido
RNF04, RNF10	Comunicação REST em JSON	Cumprido
RNF05	Dados em MySQL	Cumprido
RNF06	Arquitetura modular	Cumprido
RNF07	Código gerido no GitHub	Cumprido
RNF08	Interface baseada em protótipos Figma	Por confirmar evidências
RNF09	Interface responsiva	Por validar

2.3. FUNCIONALIDADES ADICIONAIS

Para além dos requisitos numerados, foram desenvolvidos módulos que concretizam os objetivos específicos do projeto.

Funcionalidade	**Estado**
Criação de jogos	Confirmado
Participação em jogos	Confirmado
Convites (enviar, consultar, aceitar e recusar)	Implementado, por validar
Notificações	Implementado, por validar
Histórico de jogos	Ecrã integrado; dados reais por validar
Mapa	Ecrã integrado; demonstração por validar
Check-in por QR Code	Implementado, por validar
Estatísticas	Endpoint criado; interface por validar

3. ARQUITETURA IMPLEMENTADA

A solução segue o modelo cliente-servidor. A aplicação nunca acede diretamente à base de dados: todos os pedidos passam pela API, que centraliza a autenticação, as validações e as regras de negócio.

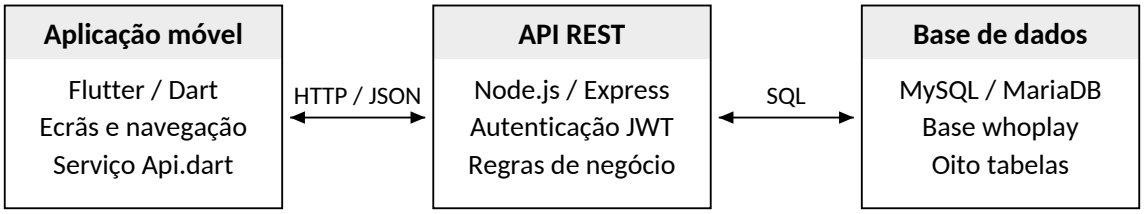


Figura 1 – Arquitetura da solução

O desenvolvimento decorreu em ambiente local, em Windows, com o VS Code. O backend corre na porta 3000 e a base de dados é servida através do XAMPP. A aplicação é executada em Flutter Web no Chrome (*localhost:3000*) e tem configuração para o emulador Android (*10.0.2.2:3000*), sendo o endereço escolhido conforme a plataforma.

4. BASE DE DADOS

A base de dados relacional **whoplay** está em funcionamento, o que foi verificado através da rota de saúde da API. O modelo tem oito tabelas e usa chaves primárias e estrangeiras para garantir a integridade das relações. As relações muitos-para-muitos são resolvidas por tabelas associativas.

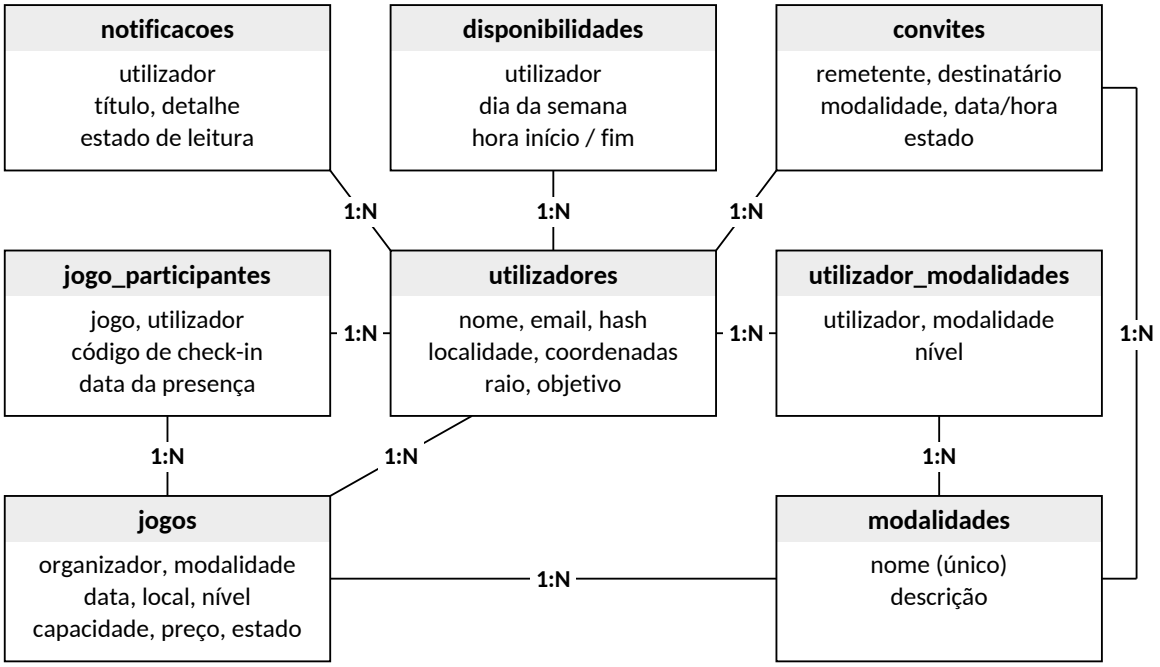


Figura 2 – Modelo relacional simplificado (atributos principais)

4.1. TABELAS

Tabela	Finalidade
utilizadores	Conta, perfil, localização, raio de procura e objetivo
modalidades	Catálogo de modalidades desportivas
utilizador_modalidades	Modalidades de cada utilizador e respetivo nível (N:N)
disponibilidades	Intervalos horários semanais de cada utilizador
jogos	Partidas criadas, com organizador, data, local, capacidade, preço e estado
jogo_participantes	Participantes de cada jogo e dados de check-in (N:N)
convites	Convites entre utilizadores e respetivo estado
notificacoes	Avisos dirigidos aos utilizadores e estado de leitura

4.2. SCRIPTS SQL

A base de dados é acompanhada por três scripts: **create.sql**, para criar a estrutura; **populate.sql**, para inserir dados fictícios de demonstração; e **queries.sql**, com as consultas relevantes. A sua execução integral numa base de dados limpa será validada antes da versão final.

5. API REST

O servidor Express configura o CORS e a leitura de JSON, regista as rotas e centraliza o tratamento de erros. As rotas de autenticação e de modalidades são públicas; as restantes estão protegidas por um middleware que verifica o token JWT enviado pela aplicação. O acesso à base de dados é feito com consultas parametrizadas.

5.1. MÓDULOS

Módulo	Responsabilidade
auth.js	Registo e autenticação
modalidades.js	Consulta do catálogo de modalidades
perfil.js	Perfil, modalidades, níveis e disponibilidade
jogadores.js	Pesquisa e consulta de jogadores
jogos.js	Criação, consulta, participação, histórico e estatísticas
convites.js	Envio, consulta, aceitação e recusa de convites
avisos.js	Consulta e gestão de notificações

5.2. PRINCIPAIS ENDPOINTS

Método	Rota	Utilização
GET	/saude	Verificar a API e a ligação à BD
POST	/auth/registo	Criar conta
POST	/auth/login	Iniciar sessão
GET	/modalidades	Listar modalidades
GET, PUT	/perfil	Consultar ou atualizar o perfil
PUT	/perfil/modalidades	Atualizar modalidades e níveis
PUT	/perfil/disponibilidade	Atualizar disponibilidade semanal
GET	/jogadores	Pesquisar jogadores
GET, POST	/jogos	Consultar ou criar jogos
GET	/jogos/historico	Consultar o histórico de jogos
GET	/jogos/estatisticas	Consultar estatísticas
GET, POST	/convites	Consultar ou enviar convites
POST	/convites/:id/aceitar	Aceitar convite
POST	/convites/:id/recusar	Recusar convite

6. APLICAÇÃO MÓVEL

O código Flutter está organizado por responsabilidades. Os ecrãs encontram-se em *lib/screens*, os componentes reutilizáveis em *lib/widgets*, os modelos em *lib/data/models.dart* e o tema em *lib/theme*. O serviço *lib/services/api.dart* concentra os pedidos HTTP, o envio do token, os tempos limite e o tratamento de erros.

6.1. NAVEGAÇÃO

A barra inferior foi simplificada de sete para cinco posições, agrupando funcionalidades relacionadas:

- **Explorar:** separadores Jogadores e Mapa;
- **Jogos:** separadores Disponíveis e Histórico;

- **Criar:** botão central para criar um jogo;
- **Inbox:** convites e notificações;
- **Perfil:** consulta e edição do perfil.

6.2. MELHORIAS E CORREÇÕES

- Criação e integração do ecrã de histórico, com filtros por jogos organizados ou participados;
- Correção do formulário de edição de perfil;
- Correção do overflow visual no ecrã de login;
- Correção de imports e parâmetros de navegação no ecrã de jogos.

7. MODELO DE COMPATIBILIDADE

A compatibilidade ordena os jogadores numa escala até 100 pontos, com cinco critérios ponderados. A distância é calculada com a fórmula de Haversine, a partir das coordenadas dos utilizadores e do raio médio da Terra (6 371 km).

Critério	**Peso**	**O que avalia**
Modalidade	30	Existência de uma modalidade em comum
Nível	25	Proximidade do nível de experiência
Disponibilidade	20	Sobreposição dos intervalos semanais
Distância	15	Proximidade geográfica
Objetivo	10	Orientação amigável, competitiva ou de treino

8. TESTES E VALIDAÇÃO

Os seguintes cenários foram executados durante o desenvolvimento.

Cenário	**Resultado**
Login	Sucesso
Listagem de jogadores a partir da API	Sucesso
Listagem de jogos	Sucesso
Criação de jogo	Sucesso
Participação num jogo	Sucesso
Edição de perfil	Sucesso
Navegação em cinco posições	Sucesso
Persistência após fechar e reabrir a aplicação	Sucesso
GET /saude (API e base de dados)	Operacional
flutter build web --release	Compilação concluída
flutter analyze	0 erros, 0 warnings, 18 sugestões informativas

Estes resultados confirmam a integração entre a aplicação, a API e a base de dados nos cenários indicados, mas não substituem uma validação completa de todas as rotas, permissões e casos de erro.

9. PRÓXIMOS PASSOS

- Validar convites, notificações, histórico, mapa, check-in QR e estatísticas;
- Registar os guiões de teste, com resultados obtidos e capturas de ecrã;
- Validar os scripts SQL numa base de dados limpa;
- Completar a documentação REST, com parâmetros, exemplos e códigos HTTP;
- Confirmar a publicação do commit *6ed695d* e reorganizar o repositório em *05_Artefactos*;
- Concretizar o objetivo de aplicação de estatística e de um método numérico.

CONCLUSÃO

Desde a primeira entrega, o WhoPlay deixou de ser apenas uma proposta e passou a ser uma solução integrada. A aplicação Flutter comunica com uma API REST e com uma base de dados MySQL de oito tabelas, e os fluxos essenciais de autenticação, procura de jogadores, criação e participação em jogos e edição de perfil funcionam.

A etapa seguinte concentra-se em validar os módulos já implementados, concluir a documentação técnica e organizar o repositório, de forma a consolidar a versão alfa e preparar a evolução da aplicação.